

РАЗРАБОТКА СИСТЕМЫ ДЛЯ ПРОКСИРОВАНИЯ И МОНИТОРИНГА ИМИТАЦИОННЫХ СЕРВИСОВ В СРЕДЕ НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ

Студент:
Медведев Клим Николаевич,
группа 221-329.
Научный руководитель:
Кружалов Алексей Сергеевич,
старший преподаватель.

Актуальность

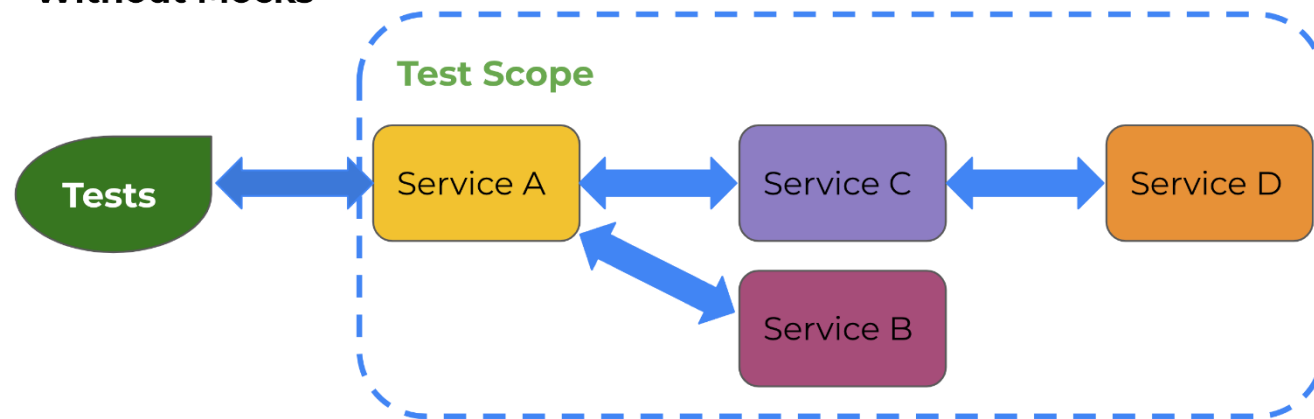
При нагрузочном тестировании сложных систем требуются десятки имитационных сервисов (заглушек). Ручное управление ими — трудоёмко и ненадёжно.

Проблемы существующих решений:

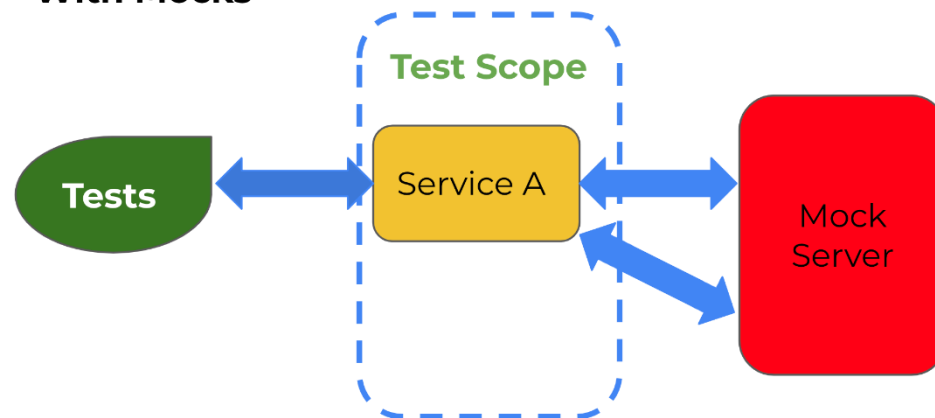
- WireMock, MockServer, Mountebank не управляют инфраструктурой JAR/WAR-процессов;
- Apache Tomcat избыточен и не поддерживает динамический Rate Limiting;
- Все зарубежные решения косвенно совместимы с ОС Astra Linux;

Вывод: ни одно из существующих решений не решает задачу централизованного управления распределённой инфраструктурой mock-сервисов в условиях импортозамещения.

Without Mocks



With Mocks



Цель и задачи работы

Цель: спроектировать и разработать автоматизированную информационную систему для централизованного управления имитационными сервисами в среде нагрузочного тестирования и мониторинга их состояния.

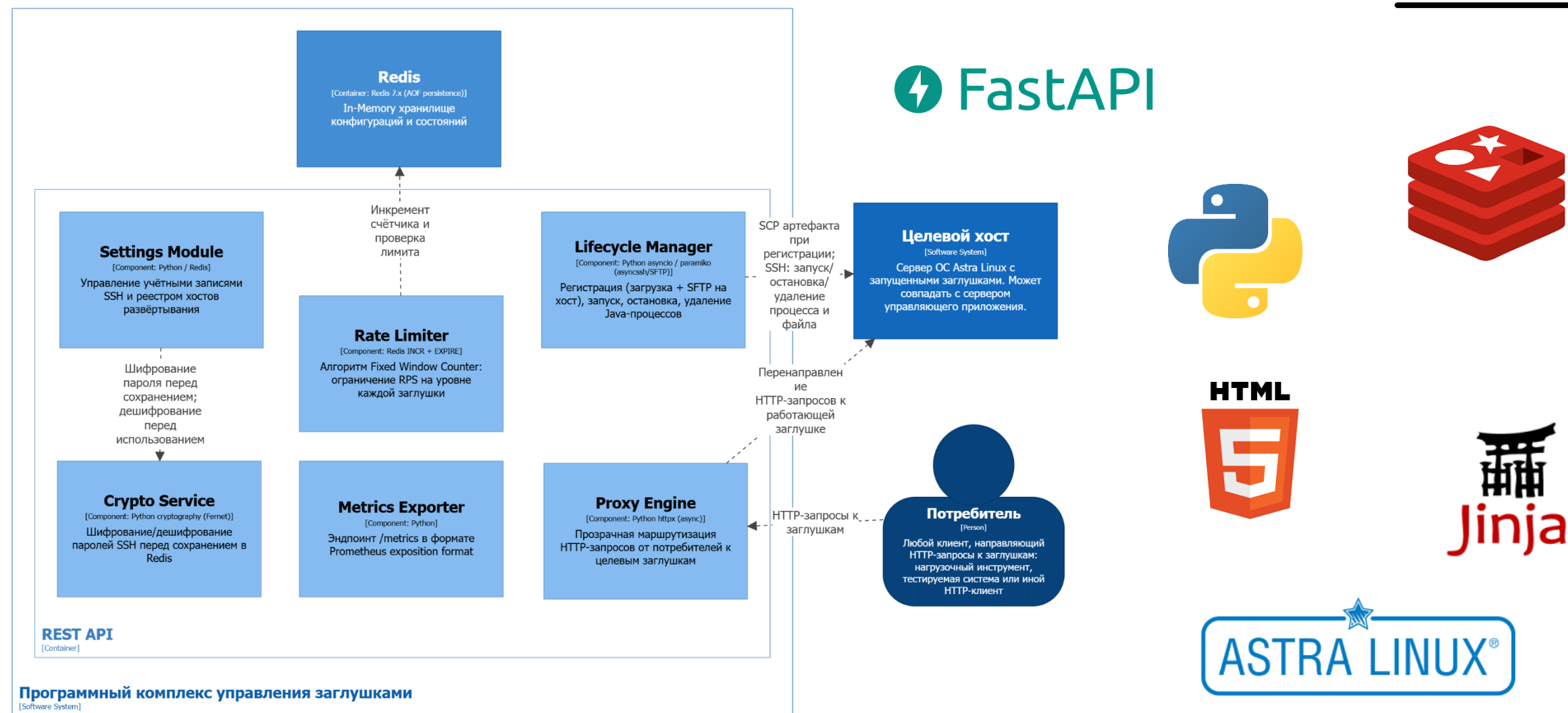
Задачи:

1. Провести анализ предметной области и существующих подходов.
2. Спроектировать архитектуру программного комплекса.
3. Обосновать выбор технологического стека.
4. Разработать алгоритмы проксирования, Rate Limiting и API управления.
5. Реализовать систему хранения конфигураций и контроля состояний.
6. Провести тестирование в среде ОС Astra Linux.

Сравнительный анализ существующих решений

Критерий	WireMock	MockServer	Apache Tomcat	Разраб. система
Управление JAR/WAR	Нет	Нет	Да	Да
Централизованная оркестрация	Нет	Нет	Частично	Да
Динамический Rate Limiting	Нет	Нет	Нет	Да
Потребление ресурсов	Высокое (JVM)	Высокое (JVM)	Высокое	Низкое
Совместимость с Astra Linux	Частично	Частично	Частично	Да

Архитектура и технологический стек



Структуры данных

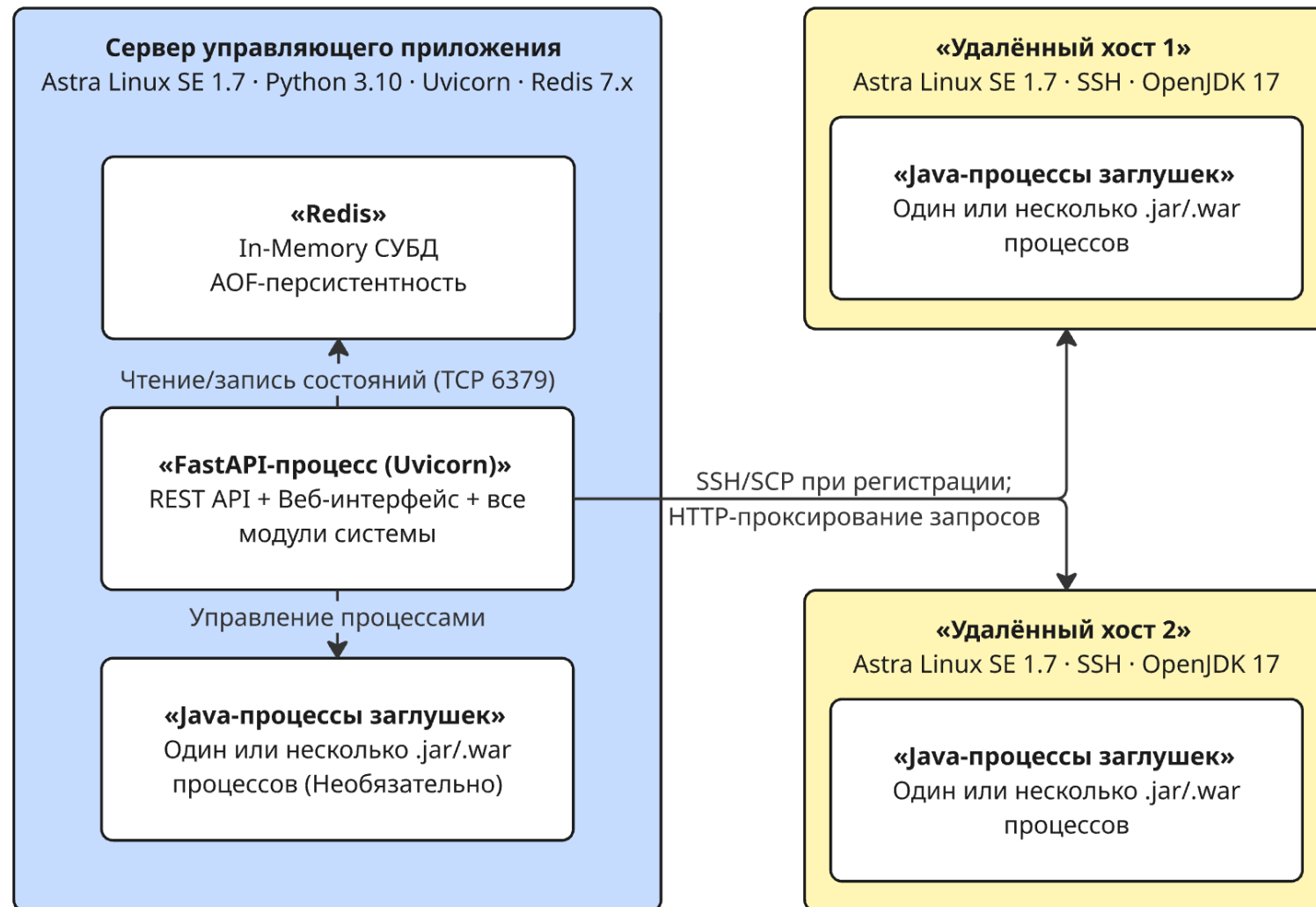
Ключ	Тип	Назначение
mocks:{filename}	Hash	Конфигурация и состояние заглушки
mocks:registry	Set	Множество имён файлов всех заглушек
hosts:{hostname}	Hash	Конфигурация целевого хоста
hosts:registry	Set	Множество hostname всех хостов
accounts:{uuid}	Hash	Учётная запись SSH
accounts:registry	Set	Множество UUID учётных записей
rate:{filename}:{window}	String	Счётчик Rate Limiter (с TTL)
settings:global	Hash	Глобальные параметры приложения



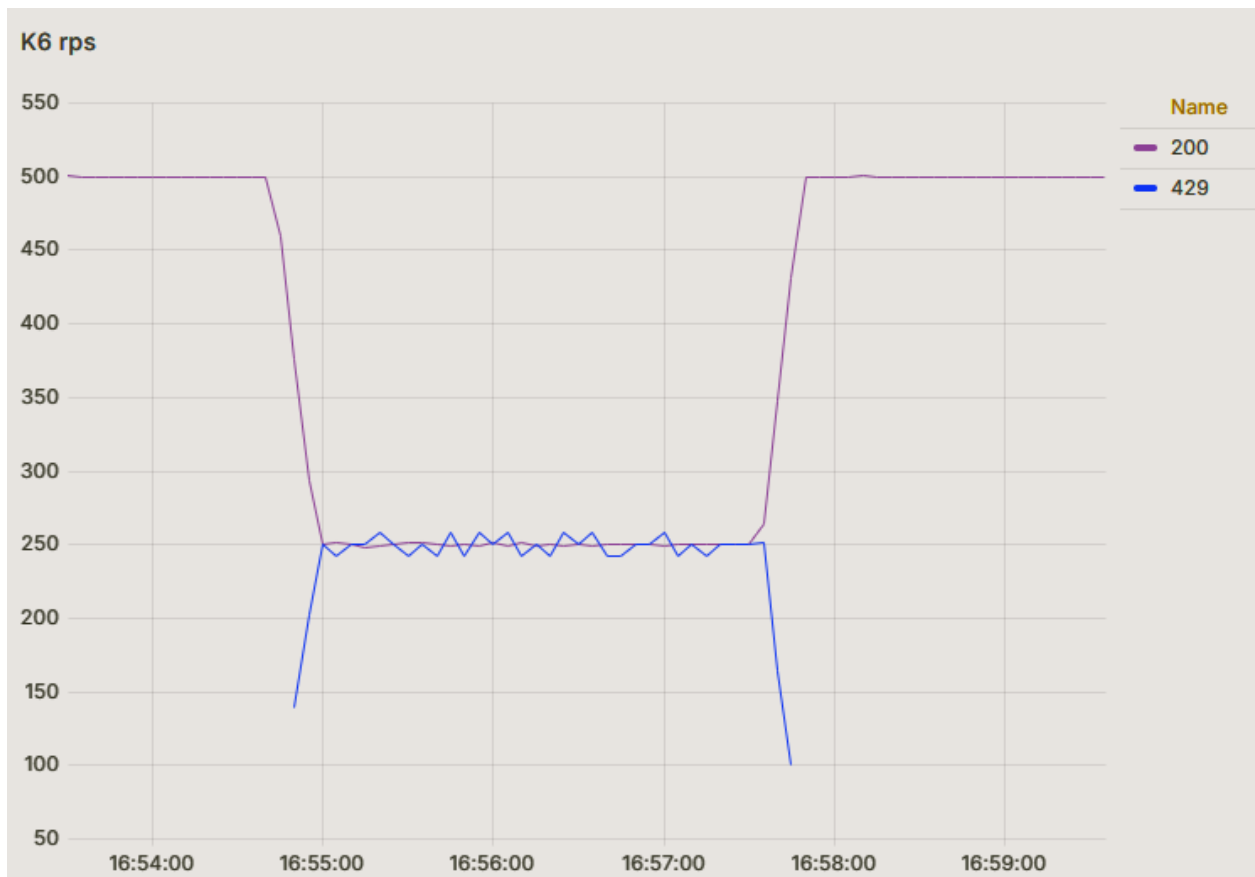
Топология развёртывания

Система поддерживает два режима без изменения кодовой базы:

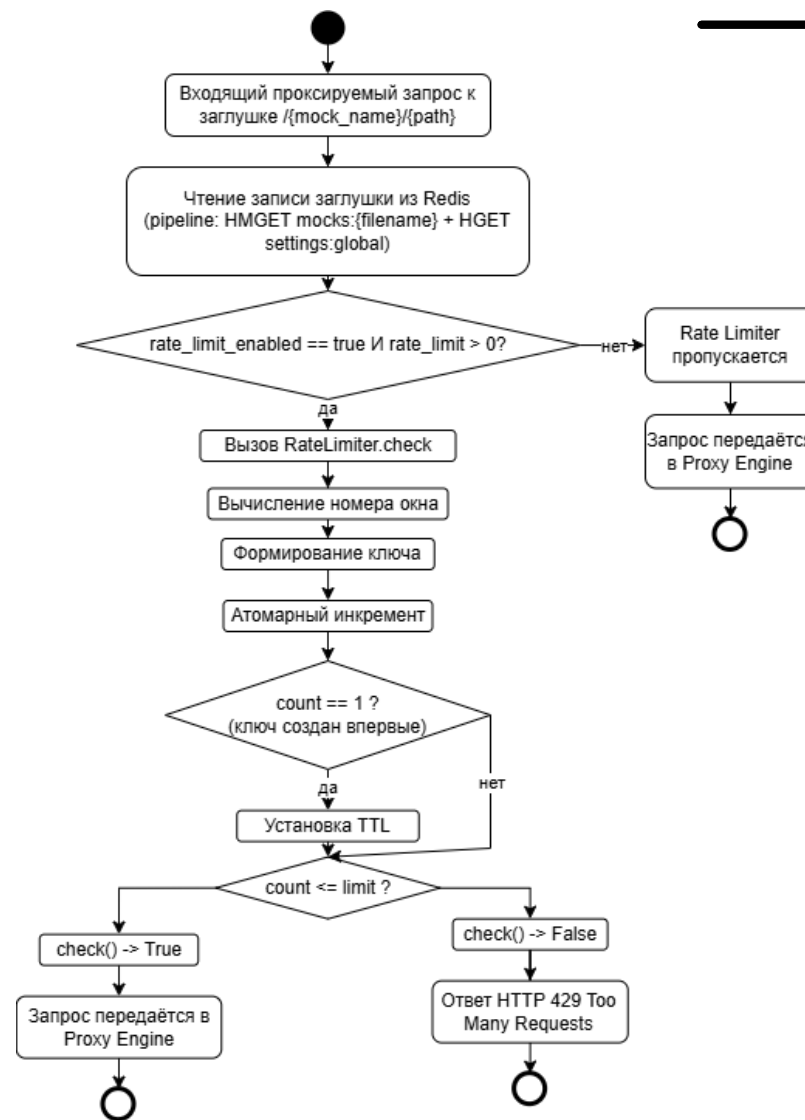
- **Cluster Mode:** один управляющий узел координирует несколько удалённых хостов через SSH/SFTP;
- **Standalone Mode:** все компоненты на одном сервере. На целевых хостах не требуется установка агентов — только SSH-сервер и OpenJDK 17.



Проксирование и Rate Limiting

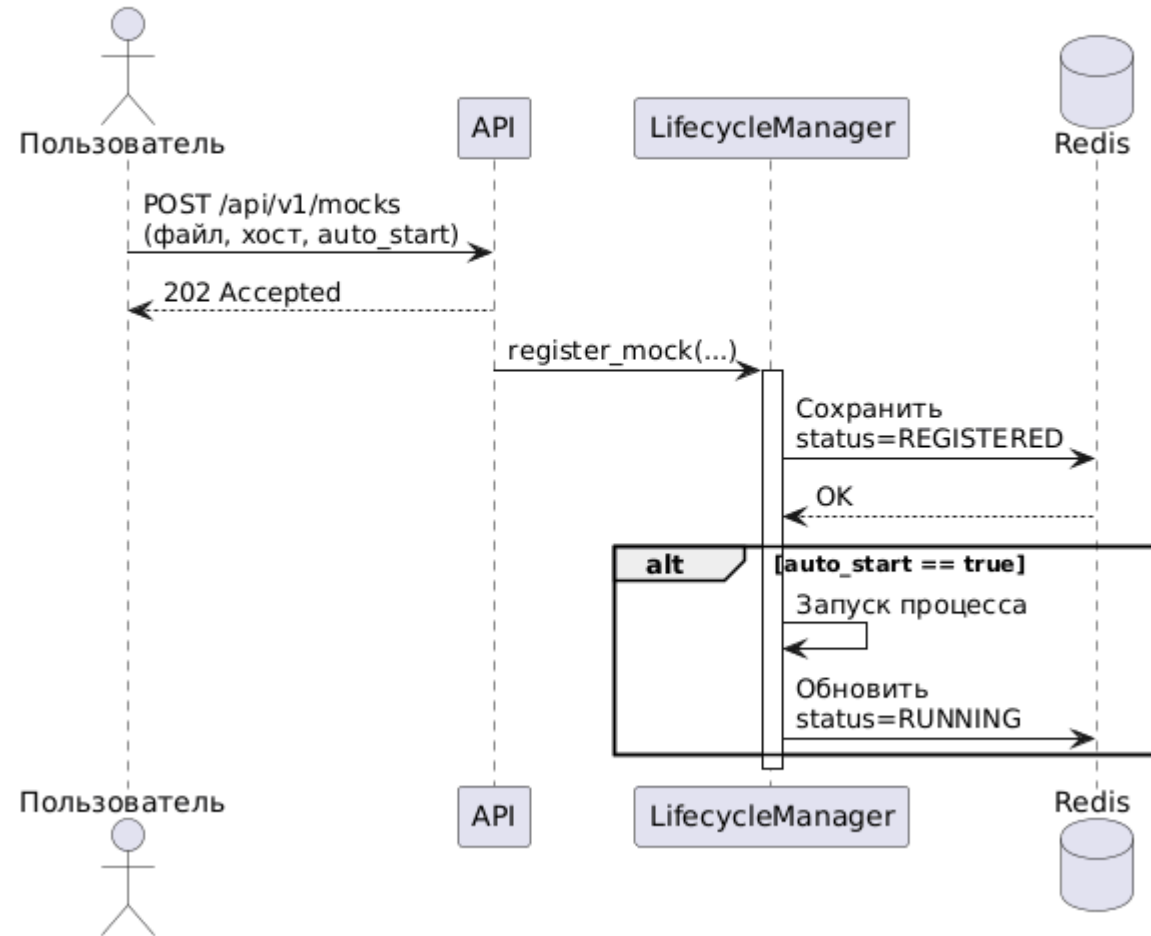


ось X — "Время", ось Y — "Интенсивность кодов ответа (RPS)"



Управление жизненным циклом

Жизненный цикл заглушки



Веб-интерфейс

Регистрация сервиса

Артефакт (.jar / .war, до 512 МБ)

Перетащите файл сюда или нажмите для выбора

Целевой хост

Выберите хост...

JVM-аргументы

-Xms128m -Xmx256m

Лимит запросов (req/s)

500

0 — без ограничений

☐

 Запустить сразу после регистрации

Отмена

Зарегистрировать

Dashboard

Redis: подключён

Сервисы

+ Зарегистрировать сервис

ВСЕГО СЕРВИСОВ

1

ЗАПУЩЕНО

1

ОШИБОК

0

ХОСТОВ ДОСТУПНО

2

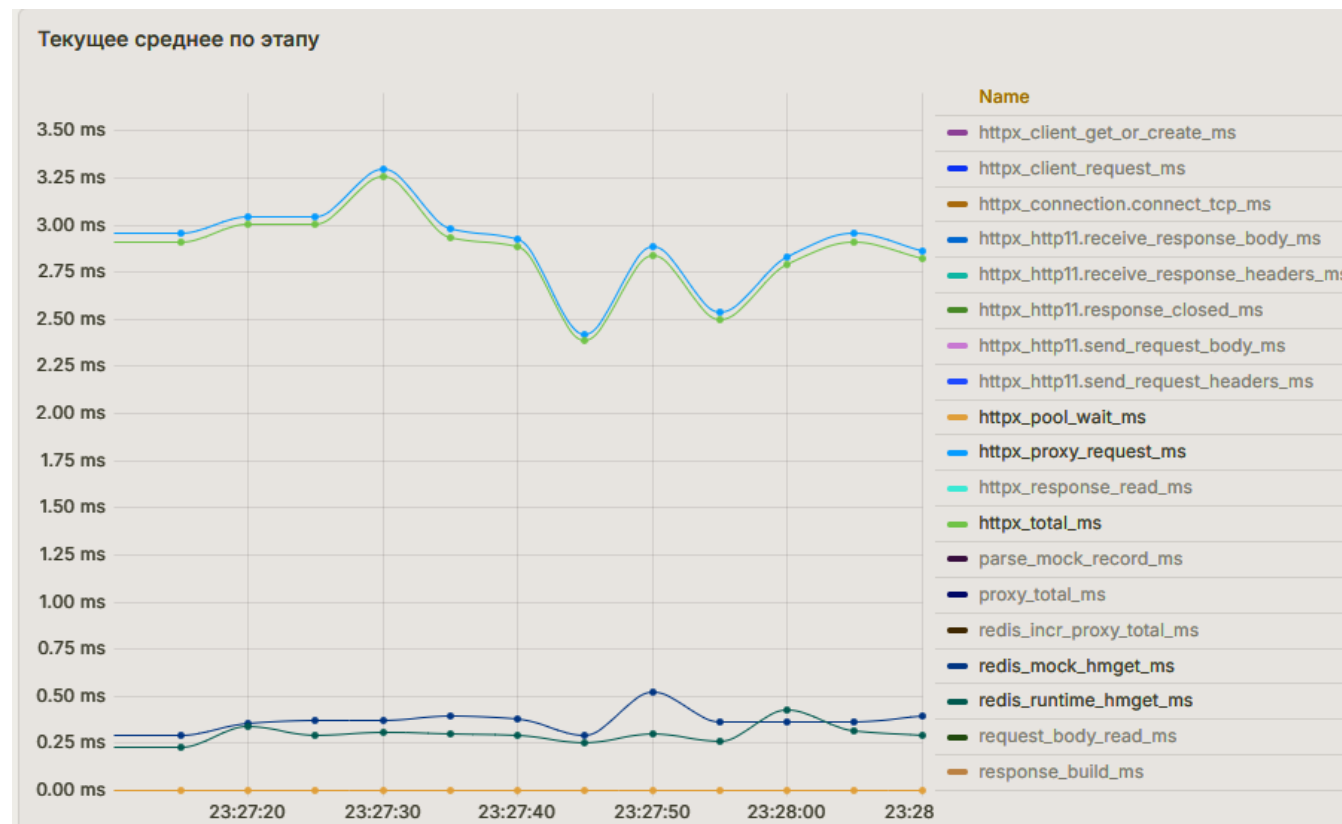
ИМЯ ФАЙЛА	ХОСТ	СТАТУС	ПОРТ	RATE LIMITER	ДЕЙСТВИЯ
stub.war	192.168.1.77	RUNNING	8101		Стоп Удалить

Сценарий	Успешность	Результат
Перезапуск при работающих процессах		Статусы RUNNING сохранены
Перезапуск при завершённых процессах		Автоматический перезапуск mosk-сервисов
Восстановление после аварии Redis		Данные восстановлены из AOF-журнала
Обнаружение сбоя планировщиком HealthChecker		Статус ERROR за ≤ 30 с
Автовозврат из ERROR в RUNNING		Статус RUNNING за ≤ 30 с

Нагрузочное тестирование

Инструмент: k6. Наблюдение через эндпоинт /metrics (Prometheus).

Сценарий	Показатель	Результат
Базовая нагрузка (10 RPS)	proxy_total_ms	3,90 мс
Нарастающая нагрузка (500 RPS)	proxy_total_ms	3,01 мс — линейный рост
Нарастающая нагрузка (1000 RPS)	proxy_total_ms	10,52 мс — без отказов
Rate Limiting (лимит 250, нагрузка 500 RPS)	Отклонение от лимита	0,13%



ось X — "Время", ось Y — "Задержка (мс)"

Результаты работы

Все 6 задач выполнены в полном объёме.

Разработан программный комплекс, не имеющий прямых аналогов среди существующих решений.

Ключевые характеристики:

- Задержка прокси: $\leq 3,01$ мс при нагрузке до 500 RPS;
- Точность Rate Limiting: отклонение 0,13% от теоретического лимита;
- Время обнаружения сбоя: ≤ 30 с;
- Впервые реализовано централизованное управление с индивидуальным Rate Limiting и агентно-независимым SSH/SCP-взаимодействием без ПО на целевых узла;
- Направления развития: динамическая маршрутизация по содержимому запроса, ролевая модель доступа, поддержка не-JVM артефактов;

**РАЗРАБОТКА СИСТЕМЫ ДЛЯ ПРОКСИРОВАНИЯ И
МОНИТОРИНГА ИМИТАЦИОННЫХ СЕРВИСОВ В СРЕДЕ
НАГРУЗОЧНОГО ТЕСТИРОВАНИЯ**

Студент:
Медведев Клим Николаевич,
группа 221-329.
Научный руководитель:
Кружалов Алексей Сергеевич,
старший преподаватель.